

GEMV 0: Basic CSL Syntax

Contents

- [Types](#)
- [Functions](#)
- [Conditional Statements and Loops](#)
- [code.csl](#)

This example is the first in a series of successive example programs demonstrating CSL and the SDK by implementing a general matrix-vector product, or GEMV.

We start by illustrating the syntax of some of CSL's core language constructs. The code in this example is not a complete program, but it shows some of the most commonly used CSL features.

CSL's syntax is like that of [Zig](#). Despite the similarity, both the purpose and the implementation of the CSL compiler are different from that of the Zig compiler.

Types

CSL includes some basic types:

- `bool` for boolean values
- `i16` and `i32` for 16- and 32-bit signed integers
- `u16` and `u32` for 16- and 32-bit unsigned integers
- `f16` and `f32` for 16- and 32-bit IEEE-754 floating point numbers

In addition to the above, CSL also supports array types and pointer types. Their use will be illustrated in subsequent examples.

Functions

Functions are declared using the `fn` keyword. The compiler provides special functions called *Builtins*, whose names start with `@` and whose implementation is provided by the compiler. All CSL builtins are described in [Builtins](#).

Conditional Statements and Loops

CSL includes `if` statements and `while` and `for` loops. These are described in greater detail in the subsequent example programs.

Note

See [GEMV Tutorial 0: Basic CSL Syntax](#) for a step-by-step walkthrough of this example.

code.csl

```
// Not a complete program; we include it here for illustrating some syntax
```

```
// Every variable must be declared either "const" or "var"  
// Const cannot be modified after declaration, but var can
```

```
// Constants defining dimensions of our matrix
```

```
const M: i16 = 4;  
const N: i16 = 6;
```

```
// 48 kB of global memory contain A, x, b, y
```

```
var A: [M*N]f32; // A is stored in row-major order  
var x: [N]f32;  
var b: [M]f32;  
var y: [M]f32;
```

```
// Initialize matrix and vectors
```

```
fn initialize() void {  
    // for loop with range syntax  
    // loops over 0, 1, ..., M*N-1  
    // idx stores the loop index  
    for (@range(i16, M*N)) |idx| {  
        // @as casts idx from i16 to f32  
        A[idx] = @as(f32, idx);  
    }  
  
    for (@range(i16, N)) |j| {  
        x[j] = 1.0;  
    }  
  
    // while loop with iterator syntax  
    var i: i16 = 0;  
    while (i < M) : (i += 1) {  
        b[i] = 2.0;  
        y[i] = 0.0;  
    }  
}
```

```
// Compute gemv
```

```
fn gemv() void {  
    for (@range(i16, M)) |i| {  
        var tmp: f32 = 0.0;  
        for (@range(i16, N)) |j| {  
            tmp += A[i*N + j] * x[j];  
        }  
        y[i] = tmp + b[i];  
    }  
}
```

```
// Call initialize and gemv functions
```

```
fn init_and_compute() void {  
    initialize();  
    gemv();  
}
```

